

1) Call by Value in C

Theory:

In Call by Value, a copy of the variable is passed to the function.

Modifications made inside the function do not affect the original variable.

Steps:

1. Define a function that takes variables as arguments.
2. Call the function with actual values.
3. Changes remain local to the function.

Syntax:

```
Void function(int a) {  
    A = a + 10;  
}
```

Example:

```
#include <stdio.h>  
  
Void update(int x) {  
    X = x + 5;  
    Printf("Inside function: x = %d\\n", x);  
}  
  
int main() {  
    Int a = 10;  
    Update(a);  
    Printf("Outside function: a = %d\\n", a);  
    Return 0;
```

```
}
```

Output:

Inside function: x = 15

Outside function: a = 10

2) Call by Reference in C

Theory:

In Call by Reference, the function receives the address of the variable.

Changes made inside the function affect the original variable.

Steps:

1. Define function with pointer parameters.
2. Use & to pass the address while calling the function.
3. Use * to modify the value at that address.

Syntax:

```
Void function(int *a) {  
    *a = *a + 10;  
}
```

Example:

```
#include <stdio.h>  
  
Void update(int *x) {  
    *x = *x + 5;  
    Printf("Inside function: x = %d\\n", *x);  
}  
  
int main() {  
    Int a = 10;  
    Update(&a);  
    Printf("Outside function: a = %d\\n", a);  
    Return 0;
```

```
}
```

Output:

Inside function: x = 15

Outside function: a = 15